

OE-EC803B — Big Data Analysis

Semester VIII • 3L:0T:0P • 3 Credits

Module 2: HDFS (Hadoop Distributed File System) & I/O

CGEC | B.Tech ECE (2023–27) | Biraj Sarkar

Contents

1	The Design of HDFS	2
2	HDFS Concepts	2
2.1	Blocks in HDFS	2
2.2	NameNode, DataNode, and Secondary NameNode	2
3	HDFS Interfaces and Command Line Interface	3
3.1	HDFS CLI (Command Line Interface)	3
3.2	Hadoop File System Interfaces	4
4	HDFS Data Flow	4
4.1	Anatomy of a File Read	4
4.2	Anatomy of a File Write	5
5	Data Ingestion: Sqoop and Flume	5
5.1	Apache Sqoop	5
5.2	Apache Flume	5
6	Hadoop Archives (HAR)	6
7	Hadoop I/O: Compression, Serialization, and File Structures	6
7.1	Compression Codecs	6
7.2	Serialization	7
7.3	File-Based Data Structures	7

1. The Design of HDFS

Hadoop Distributed File System (HDFS)

HDFS is a distributed, block-structured file system designed to store very large files (typically Gigabytes to Petabytes) reliably across a cluster of commodity servers, providing high throughput access to application data using a write-once, read-many access pattern.

Key design goals and assumptions include:

- **Commodity Hardware Failure:** HDFS assumes hardware failures are common rather than exceptional. High fault tolerance is built in through automatic block replication.
- **Large Datasets:** Optimized to support large files. A single HDFS cluster can support millions of files.
- **Streaming Data Access:** Designed for batch processing rather than low-latency interactive access. Focuses on high data throughput over low latency.
- **Simple Coherency Model:** Uses a **write-once, read-many** pattern. Once a file is written, closed, and replicated, it cannot be modified inline. Appends are supported in later versions but random edits are not.

2. HDFS Concepts

2.1 Blocks in HDFS

Unlike standard physical hard drives where disk blocks are 512 bytes or 4KB, HDFS files are divided into large, discrete chunks called **Blocks**.

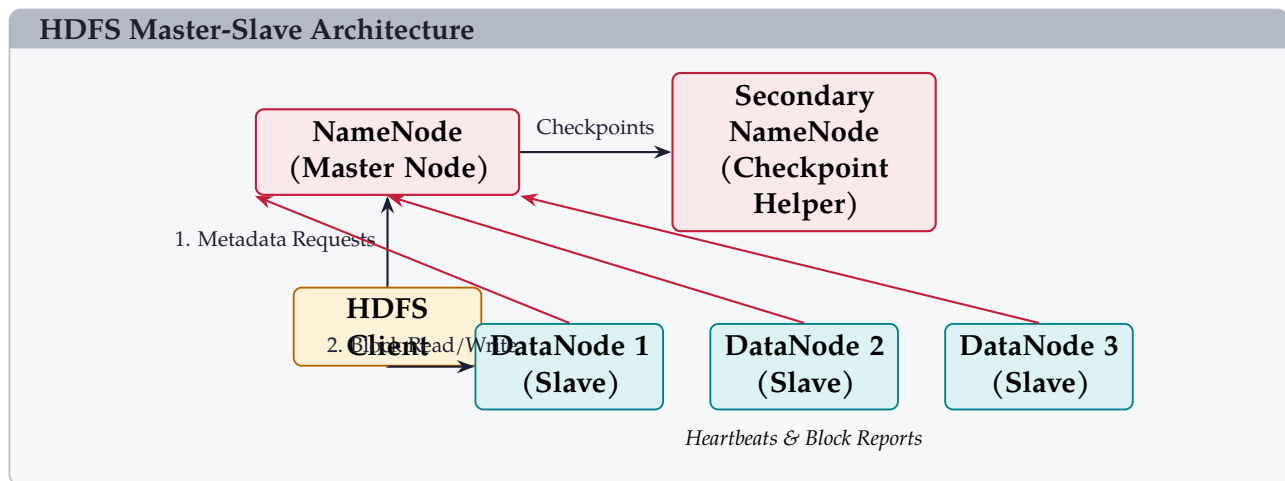
HDFS Block Size Properties

The standard block size in modern Hadoop distributions is **128MB** (or **256MB**).

- **Reason for large blocks:** To minimize the cost of disk seeks. By making blocks large, the time taken to transfer data from disk to network is significantly larger than the time taken to seek to the start of the block.
- **Block Storage:** If a file is smaller than the block size (e.g., a 10MB file), it does not consume the full 128MB of physical space; it only uses 10MB on the local disk.

2.2 NameNode, DataNode, and Secondary NameNode

HDFS uses a master-slave architecture containing three key operational nodes:



- **NameNode (Master):**

- Manages the file system namespace (directories, files).
- Tracks the mapping of files to block IDs and their physical DataNode locations.
- Stores metadata in RAM for speed. Persists namespace changes in the **Edit Log** and holds the point-in-time filesystem state in **FSImage**.

- **DataNode (Slave):**

- Stores and retrieves HDFS data blocks on the local physical filesystem as directed by the NameNode.
- Sends periodic **Heartbeats** (confirming node availability) and **Block Reports** (listing all local blocks) to the NameNode.

- **Secondary NameNode:**

- It is NOT a backup/failover NameNode.
- It periodically merges the active **Edit Log** with the **FSImage** to prevent the Edit Log from growing too large. This process is called **Checkpointing**.

3. HDFS Interfaces and Command Line Interface

3.1 HDFS CLI (Command Line Interface)

Users interact with HDFS files using standard terminal commands prefixed with 'hdfs dfs' or 'hadoop fs'.

Common HDFS Terminal Commands

Command	Description and Syntax
'hdfs dfs -ls <path>'	Lists directory contents (analogous to Unix 'ls').
'hdfs dfs -put <src> <dest>'	Copies a local file from the client host into HDFS.
'hdfs dfs -get <src> <dest>'	Retrieves a file from HDFS and stores it locally.
'hdfs dfs -cat <path>'	Displays file contents directly on stdout.
'hdfs dfs -mkdir <path>'	Creates a new directory in HDFS.
'hdfs dfs -rm <path>'	Deletes a file or folder from HDFS.

3.2 Hadoop File System Interfaces

Hadoop supports programmatic interfaces to access HDFS files:

1. **Java API:** The native interface. Uses the 'org.apache.hadoop.fs.FileSystem' abstraction class, executing methods like 'open()', 'create()', and 'delete()'.
2. **WebHDFS REST API:** Provides complete read/write HTTP REST protocol endpoints to interact with HDFS from non-Java languages without loading fat native client libraries.

4. HDFS Data Flow

Data flow operations in HDFS isolate the client from the details of block replication and network routing.

4.1 Anatomy of a File Read

1. The Client calls 'open()' on the 'FileSystem' object (which is a 'DistributedFileSystem' instance).
2. The 'DistributedFileSystem' calls the NameNode using RPC (Remote Procedure Call) to get the locations of the blocks for the first part of the file.
3. The NameNode returns a list of DataNodes that have copies of those blocks, sorted by network proximity to the client.
4. The client calls 'read()' on the input stream ('FSDataInputStream'), which connects to the nearest DataNode.
5. Once the end of the block is reached, 'FSDataInputStream' closes the connection to that DataNode and opens a connection to the nearest DataNode for the next block, until reading is complete.

4.2 Anatomy of a File Write

1. The Client calls 'create()' on the 'DistributedFileSystem' object.
2. The 'DistributedFileSystem' makes an RPC call to the NameNode to create a new file in the filesystem namespace. The NameNode checks permissions and verifies the path does not already exist.
3. The client writes data to the output stream ('FSDataOutputStream'), which splits it into packets and writes them to an internal queue called the **Data Queue**.
4. The **Data Streamer** requests the NameNode to allocate new blocks and choose DataNodes to store replicas (typically 3 DataNodes for a replication factor of 3).
5. A pipeline is formed between the selected DataNodes (e.g., DataNode A → DataNode B → DataNode C). The Data Streamer streams packets to the first DataNode, which passes them to the second, which passes them to the third.
6. When the client finishes writing data, it calls 'close()', which flushes all remaining packets to the DataNodes and contacts the NameNode to commit the file creation.

5. Data Ingestion: Sqoop and Flume

To analyze data in Hadoop, it must first be ingested from external sources.

5.1 Apache Sqoop

Apache Sqoop (SQL to Hadoop)

Sqoop is a tool designed to transfer bulk data between relational databases (RDBMS) and the Hadoop Ecosystem (HDFS, Hive, HBase). It translates import and export commands into MapReduce jobs to execute parallel data transfers.

- **Import Mechanism:** Reads rows from an RDBMS table, generates a MapReduce job where mappers read data in parallel using SQL boundaries, and writes the output files to HDFS.
- **Export Mechanism:** Reads files from HDFS and inserts records into a target database table in parallel using database insert statements.

5.2 Apache Flume

Apache Flume

Flume is a distributed service designed for efficiently collecting, aggregating, and moving large amounts of streaming log data (such as web server logs or social media streams) into HDFS.

An active Flume agent consists of three main components:

1. **Source:** Receives event data from external generators (e.g., Syslog, HTTP) and pushes it into one or more channels.

2. **Channel:** A passive store that buffers the events until they are consumed by a sink (can be in-memory for speed or file-based for durability).
3. **Sink:** Removes events from the channel and stores them in an external repository like HDFS or HBase.

6. Hadoop Archives (HAR)

Every file, directory, and block in HDFS occupies about 150 bytes of memory in the NameNode's RAM. If a cluster stores millions of small files (smaller than the block size), the NameNode RAM becomes a performance bottleneck.

Hadoop Archives (HAR)

Hadoop Archives (HAR) are file archiving tools that pack HDFS files into larger archive files ('.har' files) to reduce the metadata overhead in the NameNode, while still allowing applications to access the individual files transparently.

A '.har' archive contains index files listing the metadata and a set of part files containing the concatenated data blocks.

7. Hadoop I/O: Compression, Serialization, and File Structures

7.1 Compression Codecs

Hadoop supports data compression to save storage space and reduce network and disk I/O overhead.

Codec	Algorithm	Splittable?	Key Characteristics
Gzip	DEFLATE	No	High compression ratio, slow speed.
Bzip2	Burrows-Wheeler	Yes	Extremely high compression ratio, very slow.
LZO	LZO	Yes (if indexed)	Fast compression and decompression.
Snappy	Snappy	No	Very fast speed, moderate compression.

Why Splittability Matters: If a compression format is splittable, MapReduce can split the file and process different portions on separate mappers in parallel. If it is not splittable, a single mapper must process the entire file sequentially, breaking the parallel execution model.

7.2 Serialization

Serialization

Serialization is the process of converting structured objects into a stream of bytes for transmission over a network or storage on disk. **Deserialization** is the reverse process of turning a byte stream back into live runtime objects.

Hadoop does not use standard Java serialization because it is too bloated. Instead, it defines its own lightweight protocols:

- **Writable Interface:** Mappers and Reducers exchange keys and values implementing the **Writable** interface. It provides high-speed serialization using compact byte formats.
- **Avro:** A language-neutral serialization system that stores data schemas in JSON files, making it easy for different languages (Java, C++, Python) to share data records.

7.3 File-Based Data Structures

Hadoop provides specialized container files for binary data: 1. **SequenceFile:** A persistent, binary container file format for storing key-value pairs. It supports compression and includes **sync markers** to allow MapReduce splits. 2. **MapFile:** A sorted version of **SequenceFile** that contains an index file ('/index') alongside the data file ('/data') to allow fast random access by key.

Quick Revision — Module 2

HDFS Concepts, CLI & I/O Architectures | OE-EC803B

One-Line Definitions

- **DataNode:** A slave node in HDFS that physically stores data blocks on local disks.
- **Secondary NameNode:** A helper node that periodically merges Edit Logs with FSImage (Checkpointing).
- **Splittability:** The property of a file format/codecs that allows it to be split into chunks for parallel MapReduce execution.
- **SequenceFile:** A persistent binary file format that stores sequential key-value data.
- **Avro:** A serialization framework using JSON-defined schemas for cross-language data exchanges.

Frequently Asked / Viva Questions

Q1. Why does HDFS use a large block size (e.g., 128MB)?

Large block sizes minimize the disk seek overhead. Since disk seeks take several milliseconds, setting the block size to 128MB ensures that the disk transfer time (reading data) is significantly larger than the seek time, maximizing throughput.

Q2. How does checkpointing work in the Secondary NameNode?

The Secondary NameNode fetches the active Edit Log and current FSImage from the NameNode, merges them in memory to generate a new FSImage checkpoint, and sends it back to the NameNode. This keeps the Edit Log from growing too large.

Q3. Describe the HDFS replication strategy (Rack Awareness).

For a replication factor of 3: HDFS places the first replica on a node in the local rack, the second replica on a different node in a remote rack, and the third replica on a different node within that same remote rack. This balances fault tolerance and write latency.

Q4. What happens if a DataNode fails in an HDFS cluster?

If a DataNode fails to send Heartbeats to the NameNode for a timeout period (typically 10 minutes), the NameNode marks it as dead. It scans the metadata, identifies all blocks stored on the failed node, and schedules copies of those blocks to other nodes to restore the replication factor.

Q5. What is the difference between client-side reads and client-side writes in HDFS?

During a read, the client only queries the NameNode for block locations and reads directly from the nearest DataNode. During a write, the client streams packets to a pipeline of DataNodes, which replicate data to each other while sending ACKs back.

Frequently Asked / Viva Questions (contd.)

Q6. Explain the agent architecture of Apache Flume.

A Flume agent consists of a **Source** (ingests raw logs from external clients), a **Channel** (buffers events in memory or on disk), and a **Sink** (drains the channel to write files into HDFS).

Q7. Why are Hadoop Archives (HAR) used for small files?

Each file in HDFS consumes 150 bytes of metadata in the NameNode's RAM. Millions of small files can exhaust NameNode memory. HAR bundles these files into larger archive containers, reducing the total metadata footprint.

Q8. Which compression codecs are splittable in MapReduce?

Bzip2 is natively splittable. LZO is splittable if an index is built for it. Gzip and Snappy are not splittable.

Q9. Why did Hadoop develop its own Writable serialization system instead of using Java Serialization?

Java's default serialization is too heavy, storing class names, parent classes, and metadata along with the data. Hadoop's Writable system is lightweight, compact, and optimized for high-speed network transmission.

Q10. What is the difference between a SequenceFile and a MapFile?

A SequenceFile stores binary key-value pairs sequentially. A MapFile is a directory containing a sorted SequenceFile ('/data') and an index file ('/index') that maps keys to offsets for random lookups.

Comparison Snapshot

NameNode vs. DataNode:

Feature	NameNode	DataNode
Role	Master Node	Slave Node
Primary Data stored	File metadata, namespace mapping, block locations	Actual data blocks (replicated payload)
Storage Media	System RAM (for speed)	Physical Hard Drive / SSD
Hardware requirements	Enterprise grade (high RAM capacity)	Commodity servers (large storage disks)
Quantity in Cluster	Active/Standby pair (HA configurations)	Thousands of nodes in large clusters
Failure Impact	High (cluster down if master fails without HA)	Low (re-replicated from other nodes)